

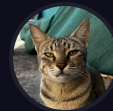
FlashInfer AI Kernel Generation Contest · MLSys 2026

Gated Delta Net (Track C): Agent Assisted

Team: Kachua · 🏆 1st Place



Romit Jain
sarvam



Pushkar Patel
NURIX

Prefill vs Decode

PREFILL

in t1 t2 t3 t4 ... tN N tokens

chunks of C

process all tokens, build state



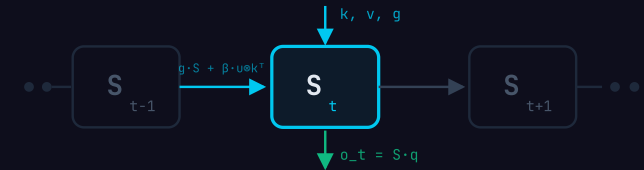
Processes the full prompt **before** generation begins

Strict causal recurrence: token t can only see $t-1, t-2, \dots$. State S at each step depends on all prior tokens — no skipping

$T = \text{seq_len}$ (hundreds to thousands of tokens), ragged batches

Challenge: expose parallelism across tokens while preserving this causal order

DECODE



$$S_t = g_t \cdot S_{t-1} + \beta_t \cdot (v_t - S_{t-1} \cdot k_t) k_t^T$$

$$o_t = q_t \cdot S_t$$



- $u = (v_t - S \cdot k_t)$ is the prediction error. k_t^T is the key direction. Their outer product is a **rank-1 matrix** — every row is a scaled copy of k_t . One direction corrected per token.

Generates **one token per call**, $T = 1$ always

State S is read, updated, and written each step

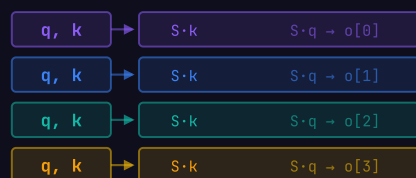
Challenge: minimize latency on a single compact instruction stream

One warp, many rows

WARP LAYOUT ON STATE S

each row i : $S[i] \cdot k$ (rank-1 update) + $S[i] \cdot q \rightarrow o[i]$ (output)
 — k and q are identical for every row

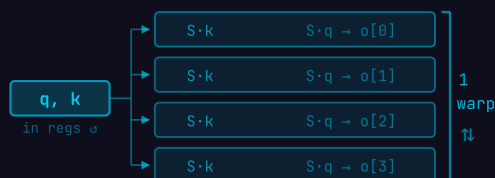
2D BLOCK - BEFORE



State S (V rows × K cols)

q, k loaded 4× — once per warp
 same data, repeated traffic

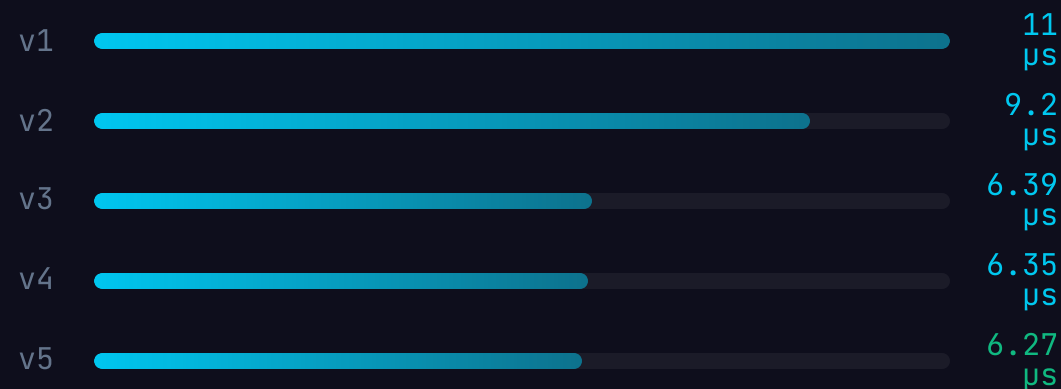
1D BLOCK - AFTER



State S (V rows × K cols)

q, k loaded once — reused per row
 stay in registers across all rows

OPTIMIZATION JOURNEY



43% faster from first naive attempt · 11 μs to **6.27 μs**

Profiling: **not bandwidth-bound** — high dependency stalls, instruction-latency regime

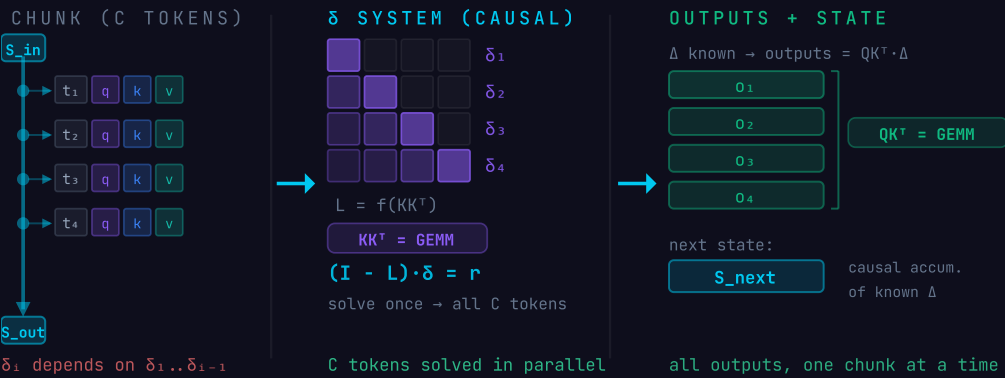
CUDA chosen for direct warp, shuffle, and scheduling control

v1–v3 hand-written; **agent loop drove v4–v5** via harness on B200

Chunks to Tensor Cores

CHUNK OF TOKENS → Δ SYSTEM → OUTPUTS

within a chunk, unknowns δ_i form a **causal lower-tri system** – solved once as a GEMM, not token-by-token



TWO AXES OF PARALLELISM



within each chunk: $(I-L) \cdot \delta = r$ solved as one GEMM

Shape-specialized **dispatch**

THREE EXECUTION PATHS

- ① **Fused chunkwise**: single kernel for small batches where two-kernel launch overhead dominates
- ② **Split-WY prepass**: two kernels for long low-batch workloads. First precomputes chunk terms (KK^T , QK^T , gates) once; second reuses them across all V-tiles
- ③ **Flat-WY**: like Split-WY but the prepass batches chunk records from *all sequences together* — fills the GPU when processing many sequences of varying lengths

WHY TRITON

Better MMA lowering on Blackwell: Triton's compiler mapped **tl.dot** to Blackwell's TCGen05 tensor core instructions more efficiently than our hand-written CUDA

Automatic pipelining: overlaps memory loads and tensor core compute without manual warp specialization

CUDA tried and abandoned — grew too complex and still couldn't match Triton's latency

OPTIMIZATION JOURNEY



58% faster from first attempt · 145 μ s to **61.5 μ s**

Human intuition, agent execution

AGENTS.md

- Edit only the **active kernel version**
- Start each experiment with a **hypothesis**
- Make **one focused change** at a time
- Validate **correctness and latency** before accepting

constrained scope

Tight Harness

- Makefile targets for local + **Modal B200** correctness checks
- Benchmarking and **NCU profiling** targets
- Scripts to parse outputs and **compare latency** across versions

objective feedback

Long-horizon Agent Loop

- Extended sessions with **no time limit** on B200
- Edit kernel → run harness → **keep improvements**, discard regressions → repeat

79 experiments

Agents were most effective with a strong human-provided baseline, explicit hypotheses, and objective harness feedback. Humans identified bottlenecks, selected directions, and caught benchmark artifacts.

THE NAME

Kachua

कछुआ · *Hindi for* tortoise

IN OUR WORK

One change at a time

Keep improvements, discard regressions

Slow is smooth, smooth is fast

Thank you!

Scan to connect with us



Code

`github.com/
romitjain/
kachua-mlsys`



Romit Jain

`sarvam.ai`



Pushkar Patel

`nurix.ai`

GDN Track · 1st Place · FlashInfer @ MLSys 2026

Acknowledgements: veitner.bearblog.dev/chunkwise-gated-delta-rule — chunkwise GDR derivation